CT6049 Data Warehousing and Data Mining

Library Data Warehouse for University Decision Support

Assignment 002 Report

Student Name	[Student Name]
Student ID	[Student ID]
Module Code	CT6049
Assessment	Assignment 002
Submission Date	[Submission Date]
Referencing Style	Harvard author-date

This report evaluates the design and implementation of a library data warehouse, its ETL process, security controls, performance features, backup and recovery approach, and a load-testing plan, in line with the CT6049 assessment brief.

Main body written to align with the 3000-word report limit. Appendices are intended to sit outside the word count.

Abstract

This report presents the design and implementation of a library data warehouse created to support university decision makers. The solution combines an operational database, a star-schema warehouse, a transactional ETL pipeline, and a secure reporting application with role-based access. The report critically evaluates the dimensional model, explains the ETL process with implementation evidence, and analyses security, user accounts, indexes, partitioning, and prepared statements. It also proposes a backup, restore, and recovery approach suitable for the deployed architecture and defines a practical load-testing plan. Harvard author-date referencing is used throughout to support the technical and design choices discussed in the report.

Contents

Abstract **i**

1 Introduction and Decision-Maker Questions **1**

2 Critical Evaluation of the Dimensional Model **2**

3 Explanation of the ETL Process **3**

4 Critical Analysis of Security, User Accounts, Indexes, Partitioning, and Prepared Statements **5**

4.1 Authentication, Authorization, and User Accounts 5

4.2 Indexes 6

4.3 Partitioning 7

4.4 Prepared Statements 7

5 Backup, Restore, and Recovery **8**

6 Load Testing Plan **8**

7 Conclusion **9**

Reference List **9**

A Application Source Artefacts **10**

B Database Build Scripts **11**

C Sample Data Scripts **11**

D ETL Scripts **11**

E Deployment Instructions **11**

F Assessment Login Details **11**

1 Introduction and Decision-Maker Questions

Data warehouses exist to organise historical data in ways that support decision-making rather than day-to-day transaction processing. In dimensional modelling, facts are centred on business events and are surrounded by descriptive dimensions that make analysis fast and business-readable (Kimball and Ross, 2013). That design principle is directly relevant to a university library, where senior stakeholders need to compare borrowing activity, overdue exposure, collection usage, and student engagement across multiple dimensions.

The aim of this project was therefore to transform a transaction-oriented library dataset into a decision-support platform for five stakeholder groups: Vice-Chancellor, Departmental Head, Admission Director, Finance Director, and Chief Librarian. The delivered system combines an operational source database, a warehouse implemented as a star schema, an ETL process, and a secure reporting application with role-based access.

The assessment brief asks for approximately ten decision-maker questions. In this implementation, those questions were translated into concrete reports and dashboards so that the warehouse design can be justified against explicit information needs rather than abstract modelling theory alone.

ID	Decision-maker question	Primary audience	Implementation evidence
Q1	Which faculties generate the highest borrowing activity and overdue pressure?	Vice-Chancellor, Departmental Head	Faculty loan and overdue views
Q2	How do borrowing volumes change over one, three, and six months by faculty or course?	Vice-Chancellor, Finance Director, Departmental Head	Monthly trend comparison with filters
Q3	Which categories and formats are most popular with students?	Chief Librarian, Admission Director	Category and peak-day reporting
Q4	How much value is tied up in collected fines, outstanding fines, and replacement risk?	Finance Director, Vice-Chancellor	Fines overview and finance dashboard
Q5	Which student years and regions show the strongest library engagement?	Admission Director, Vice-Chancellor	Demographic and regional analysis

Q6	Which students have repeated overdues or high fine balances?	Departmental Head, Admission Director, Vice-Chancellor	At-risk student reporting
Q7	How long are books typically kept by faculty?	Chief Librarian, Departmental Head, Vice-Chancellor	Average duration analysis
Q8	Which titles have never been borrowed?	Chief Librarian, Vice-Chancellor	Dead-stock reporting
Q9	When was the warehouse last refreshed and how many records were processed?	Chief Librarian, Vice-Chancellor	ETL governance and log views
Q10	Can operational and warehouse tables be inspected side by side?	Chief Librarian, Vice-Chancellor	Data-explorer views and schema traceability

2 Critical Evaluation of the Dimensional Model

The warehouse uses a star schema because its workload is overwhelmingly analytical. The central fact table, `fact_loans`, records borrowing events, while the dimensions `dim_books`, `dim_students`, and `dim_time` provide the descriptive attributes used in slicing and aggregation. This approach reflects the standard dimensional principle of declaring the grain clearly and then attaching the dimensions needed to answer business questions efficiently (Kimball and Ross, 2013).

Listing 1: Warehouse schema excerpt from server/db.ts

```
CREATE TABLE IF NOT EXISTS dim_books (
  isbn TEXT PRIMARY KEY,
  title TEXT,
  author TEXT,
  category TEXT,
  format TEXT
);

CREATE TABLE IF NOT EXISTS dim_students (
  student_id TEXT PRIMARY KEY,
  name TEXT,
  faculty TEXT,
  course TEXT,
  year INTEGER,
  region TEXT
);

```

```sql
CREATE TABLE IF NOT EXISTS fact_loans (
  fact_id INTEGER PRIMARY KEY AUTOINCREMENT,
  book_isbn TEXT,
  student_id TEXT,
  time_id INTEGER,
  loan_duration INTEGER,
  is_overdue INTEGER,
  fine_amount REAL
);
```

The strongest aspect of this model is its fitness for the decision-maker questions listed in Section 1. Borrowing is the natural business event at the centre of the warehouse, and the supporting dimensions align with how managers interpret the domain: what was borrowed, who borrowed it, and when it happened. The dedicated time dimension is especially important because it enables month, quarter, year, and day-of-week analysis without recalculating calendar attributes in every report query. This simplifies analytical SQL and reduces duplicated logic in the reporting layer.

The star schema also improves usability. A query such as "compare borrowing by faculty over the last six months" requires only one fact table and two dimensions. That is significantly easier to maintain than a more fragmented highly normalised design. For coursework-scale warehousing, this simplicity is a major practical benefit because it reduces the risk of modelling complexity overshadowing the actual information needs.

The model is not without limitations. First, the dimensions are effectively Type 1 dimensions. If a student's faculty or region changes, the ETL replaces the dimensional row rather than storing a new historical version. As a result, past facts will always reflect the latest dimension state rather than the state that existed when the event occurred. This weakens longitudinal analysis of changing student profiles. Secondly, the warehouse contains only an event-based fact table. It does not include periodic snapshot facts for holdings, daily circulation state, or stock position. That means it answers "what happened" more naturally than "what was the exact state on a date".

The choice not to snowflake the dimensions is reasonable. For the current scale, introducing separate dimensions for faculty, department, or region would add joins without materially improving the reports required by the brief. In short, the model trades some historical sophistication for clarity, speed, and implementation discipline. For this assignment, that trade-off is justified.

3 Explanation of the ETL Process

The ETL process is implemented in `server/etl.ts` and performs extract, transform, and load operations from operational source tables prefixed with `op_` into warehouse tables prefixed with `dim_` and `fact_`. The process is executed automatically on application

startup and can also be triggered manually by privileged users through the administration interface. Each run is written to `sys_etl_logs`, so refresh activity can be audited.

Listing 2: ETL control flow from server/etl.ts

```
export function runETL() {
  const startTime = new Date().toISOString();
  const result = db.prepare(
    'INSERT INTO sys_etl_logs (start_time, status, message) VALUES (?, ?, ?)'
  ).run(startTime, 'RUNNING', 'ETL Started');

  try {
    let records = 0;
    const transaction = db.transaction(() => {
      loadDimBooks();
      loadDimStudents();
      records = loadFactLoans();
    });
    transaction();
    db.prepare(
      'UPDATE sys_etl_logs SET end_time = ?, status = ?, message = ?, ' +
      'records_processed = ? WHERE log_id = ?'
    ).run(new Date().toISOString(), 'SUCCESS', 'Completed successfully',
        records,
      result.lastInsertRowid);
  } catch (error) {
    db.prepare(
      'UPDATE sys_etl_logs SET end_time = ?, status = ?, message = ? ' +
      'WHERE log_id = ?'
    ).run(new Date().toISOString(), 'FAILED', error.message, result.
        lastInsertRowid);
    throw error;
  }
}
```

The extract phase is intentionally straightforward: complete rowsets are read from the operational book, student, and loan tables. The transformation phase is where the warehouse gains analytical value. Borrow dates are expanded into a time dimension, and loan facts are enriched with measures that do not exist directly in the operational schema, such as loan duration and overdue status.

Listing 3: Transformation logic for time and loan measures

```
const date = new Date(dateStr);
const monthVal = date.getMonth() + 1;
const month = date.toLocaleString('default', { month: 'short' });
```

```
const dayOfWeek = date.toLocaleString('default', { weekday: 'long' });
const quarter = `Q${Math.floor((date.getMonth() + 3) / 3)}`;

if (loan.return_date) {
  const returnDate = new Date(loan.return_date);
  const diffTime = Math.abs(returnDate.getTime() - borrowDate.getTime());
  loanDuration = Math.ceil(diffTime / (1000 * 60 * 60 * 24));
  isOverdue = returnDate > dueDate ? 1 : 0;
} else {
  isOverdue = now > dueDate ? 1 : 0;
}
```

The load phase writes to dimensions first and facts afterwards, all within a transaction. Using a transaction is essential because it ensures that a failed load cannot leave the warehouse in a partially refreshed state. The process is therefore atomic and easier to recover operationally.

The ETL design is effective for the assignment, but it is also deliberately simple. The fact table is reloaded through a full refresh rather than an incremental load. This is acceptable for a seeded coursework dataset, yet it would be inefficient at larger scales. The current fact design also omits the operational `loan_id`, which means incremental change capture would require further schema design. A more production-oriented approach would store the source business key and apply a watermark, timestamp, or change-data-capture mechanism. The current approach was chosen because it is transparent, reproducible, and easy to verify during assessment.

4 Critical Analysis of Security, User Accounts, Indexes, Partitioning, and Prepared Statements

4.1 Authentication, Authorization, and User Accounts

Authentication is implemented with JSON Web Tokens, following the standard token format defined in RFC 7519 (Jones et al., 2015). User credentials are stored in `op_staff`, passwords are hashed with `bcryptjs`, and access to restricted routes is controlled by role checks. The use of bcrypt is a sound design choice because adaptive password hashing remains a widely accepted defence against password cracking (Provos and Mazières, 1999).

Listing 4: Authentication and RBAC evidence from server/routes.ts

```
const user = db.prepare('SELECT * FROM op_staff WHERE username = ?').get(
    username);

if (!user || !bcrypt.compareSync(password, user.password)) {
  db.prepare(
```

```
    'INSERT INTO sys_auth_logs (username, timestamp, success, ip_address) ' +
    'VALUES (?, ?, 0, ?)'
  ).run(username, new Date().toISOString(), ip);
  return res.status(401).json({ message: 'Invalid credentials' });
}

const token = jwt.sign({ id: user.staff_id, role: user.role }, JWT_SECRET,
  { expiresIn: '24h' });
```

This design satisfies the brief's requirement for secure authentication and authorisation because it demonstrates separate user accounts, role-based access control, and auditable login activity. The application also records both successful and failed logins in `sys_auth_logs`, which strengthens governance evidence.

The weaknesses are mainly those expected in an academic prototype. The seeded accounts all use the same demonstration password, and the application exposes demo credentials to support marking. That is convenient for assessment, but it would be unacceptable in production. In addition, the code contains a development fallback for the JWT secret if an environment variable is not present. In a real deployment, secret management should be mandatory rather than permissive. These limitations should not be hidden; they are part of a balanced critical evaluation.

4.2 Indexes

Indexes were added explicitly to match the main aggregation paths in the reporting workload.

Listing 5: Index definitions from server/db.ts

```
CREATE INDEX IF NOT EXISTS idx_fact_loans_book ON fact_loans(book_isbn);
CREATE INDEX IF NOT EXISTS idx_fact_loans_student ON fact_loans(student_id);
CREATE INDEX IF NOT EXISTS idx_fact_loans_time ON fact_loans(time_id);
CREATE INDEX IF NOT EXISTS idx_dim_time_year_month ON dim_time(year, month_val)
    ;
CREATE INDEX IF NOT EXISTS idx_dim_students_faculty ON dim_students(faculty);
CREATE INDEX IF NOT EXISTS idx_dim_books_category ON dim_books(category);
```

These indexes are well aligned with the implemented reports. Time-based trends rely on `time_id` and the year-month combination, faculty summaries rely on `faculty`, and category reports rely on `category`. This is a sensible design because indexing should support proven query patterns rather than be added indiscriminately.

That said, the indexing strategy remains conservative. It would be possible to introduce more composite indexes once query plans demonstrate a consistent need, for example on combinations used by filterable trend reports. The current approach is appropriate

because the dataset is moderate and the assessment emphasis is on justified optimisation rather than maximal index count.

4.3 Partitioning

Partitioning was evaluated as part of the performance analysis but was not physically implemented. This decision follows from the selected platform. SQLite is a strong fit for a self-contained coursework prototype, yet it does not provide enterprise-style native table partitioning in the same way as platforms such as PostgreSQL or SQL Server. The design therefore uses logical time organisation through a dedicated time dimension and supporting indexes rather than physical partitions.

For the current data volume, that decision is acceptable. Queries over monthly and yearly borrowing trends remain practical because the data size is limited and because time-based filtering is already supported by the dimensional design. However, the absence of physical partitioning would become a more significant weakness if the system were required to retain substantially more historical data. In that scenario, year-based range partitioning of the fact table would improve maintenance, backup management, and time-bounded scans. The current solution should therefore be understood as fit-for-purpose rather than universally scalable.

4.4 Prepared Statements

Prepared statements are used throughout the implementation via `better-sqlite3`. This is good practice for both security and performance. Parameterised queries reduce the risk of SQL injection, a control recommended by OWASP, and they also allow repeated SQL statements to be reused efficiently (OWASP Foundation, n.d.-a,n).

Listing 6: Prepared statement examples from ETL and reporting code

```
const insertDim = db.prepare(
  'INSERT OR REPLACE INTO dim_students ' +
  '(student_id, name, faculty, course, year, region) VALUES (?, ?, ?, ?, ?, ?)'
);

const user = db.prepare(
  'SELECT * FROM op_staff WHERE username = ?'
).get(username);
```

Prepared statements are especially valuable in this project for two reasons. First, ETL inserts are repetitive and benefit from statement reuse. Secondly, several routes accept user-controlled inputs such as usernames, filters, and query parameters, making injection prevention essential. One route does assemble SQL with a dynamic table name in order to support the data explorer. That route remains defensible because the table identifier is constrained by an explicit allow-list before the statement is created. Nevertheless,

this should be treated as an exception to the general rule that identifiers must not be interpolated from uncontrolled input.

5 Backup, Restore, and Recovery

The application uses SQLite in Write-Ahead Logging mode, which improves concurrency and supports online-safe backup approaches (SQLite Consortium, n.d.-a). The most suitable backup strategy is therefore a scheduled copy created through the SQLite online backup mechanism exposed by the `better-sqlite3` library, or an equivalent controlled file-level backup process consistent with SQLite guidance (SQLite Consortium, n.d.-b).

A suitable operational policy is:

- create a nightly timestamped backup of `library_warehouse.db`;
- store backup files outside the live application directory;
- retain short-term daily copies and a smaller number of weekly copies;
- validate that a backup can be opened before marking it as successful.

Recovery should remain simple and procedural:

1. stop the application to prevent further writes;
2. preserve the failed database files for diagnosis;
3. restore the latest verified backup to the live database location;
4. restart the application;
5. verify login, warehouse counts, and ETL log visibility.

This strategy is proportionate to the architecture. A single-node coursework application does not require a complex replicated backup topology, yet it still benefits from a disciplined restore process. The design is therefore operationally realistic without becoming unnecessarily elaborate.

6 Load Testing Plan

The purpose of load testing is to confirm that the warehouse-backed application remains stable when multiple decision makers use the reporting interface concurrently. The most meaningful tests are therefore those that target authentication and report endpoints rather than static assets alone.

A practical test plan is:

1. simulate a login burst against `/api/auth/login`;
2. simulate repeated analytical reads against the core warehouse endpoints:
   - `/api/reports/monthly-trends`
   - `/api/reports/loans-by-faculty`
   - `/api/reports/fines-overview`
   - `/api/app/overview`
3. include a low-frequency privileged request to `/api/etl/run` to confirm that manual refresh activity does not destabilise normal reads.

The recommended profile is 50 concurrent virtual users distributed across the available roles. The test should run with a clear ramp-up period, a sustained workload window, and a short cool-down phase. The following success criteria are appropriate for this coursework deployment:

- 95% of report requests complete within 500 ms on the assessment machine;
- no unhandled server errors occur during the run;
- authentication failures occur only for deliberately invalid credentials;
- the application remains responsive after an ETL refresh request.

This plan is intentionally scoped to the system that has actually been built. It does not pretend to be an enterprise benchmark. Instead, it measures whether the implemented warehouse, security layer, and reporting endpoints perform reliably under a realistic academic demonstration workload.

7 Conclusion

This project satisfies the design intent of the CT6049 assessment brief by implementing a complete warehouse-oriented decision-support application for a university library. The dimensional model is appropriate for the required analytics workload, the ETL process is explicit and auditable, and the application demonstrates secure access control, warehouse-focused reporting, performance-minded indexing, and operational governance features.

The evaluation has also identified important constraints. The ETL process is a full refresh rather than an incremental pipeline, dimensions are not historised, physical partitioning is not implemented, and several security settings remain appropriate only for demonstration. These are legitimate limitations, but they do not undermine the coherence of the submission. On the contrary, they show that the implementation has been designed with clear scope boundaries and with design choices that can be defended against the assessment brief.

Reference List

References

Jones, M., Bradley, J. and Sakimura, N. (2015) *JSON Web Token (JWT)*. RFC 7519. Available at: https://www.rfc-editor.org/rfc/rfc7519 (Accessed: 13 March 2026).

Kimball, R. and Ross, M. (2013) *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3rd edn. Indianapolis, IN: John Wiley & Sons.

OWASP Foundation (n.d.-a) *SQL Injection Prevention Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (Accessed: 13 March 2026).

OWASP Foundation (n.d.-b) *Query Parameterization Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html (Accessed: 13 March 2026).

Provos, N. and Mazières, D. (1999) 'A future-adaptable password scheme', in *Proceedings of the 1999 USENIX Annual Technical Conference*. Monterey, CA, 6–11 June 1999. Berkeley, CA: USENIX Association.

SQLite Consortium (n.d.-a) *Write-Ahead Logging*. Available at: https://www.sqlite.org/wal.html (Accessed: 13 March 2026).

SQLite Consortium (n.d.-b) *Online Backup API*. Available at: https://www.sqlite.org/backup.html (Accessed: 13 March 2026).

A Application Source Artefacts

The assessment brief requests source code as appendix material. This implementation is delivered in TypeScript rather than Java, so the equivalent source artefacts are:

- backend entry point: `server/index.ts`
- database schema and indexes: `server/db.ts`
- ETL implementation: `server/etl.ts`
- middleware and security controls: `server/middleware.ts`
- API routes: `server/routes.ts`
- frontend entry point: `src/index.tsx`
- main application shell: `src/App.tsx`

- reporting interface: `src/components/Reports.tsx`
- administration interface: `src/components/Admin.tsx`

B Database Build Scripts

Operational and warehouse tables are built from the following project artefacts:

- `server/db.ts` for schema creation and index creation;
- `package.json` for startup and build commands;
- `README.md` for execution instructions.

C Sample Data Scripts

Sample operational data is generated by `server/seed.ts`. This file inserts:

- staff accounts and roles;
- books with category, format, and price attributes;
- students with faculty, course, year, and region attributes;
- 10,000 loan rows;
- budget data for the finance dashboard.

D ETL Scripts

The ETL process is implemented in `server/etl.ts`. Automatic execution occurs on startup in `server/index.ts`, while manual execution and ETL history are exposed through routes defined in `server/routes.ts`.

E Deployment Instructions

The coursework deployment sequence is:

1. install dependencies with `npm install`;
2. start the application with `npm run dev`;
3. open the local frontend URL displayed by Vite;
4. log in using one of the assessment accounts listed below.

F Assessment Login Details

- Vice-Chancellor: `vc` / `password`
- Departmental Head: `dh` / `password`
- Finance Director: `fd` / `password`
- Chief Librarian: `cl` / `password`
- Admission Director: `ad` / `password`